SQL Lab Record

Labs 7 - 10: Joins, Grouping, Procedures and Triggers

LAB 7 | Joins

Setup

```
CREATE TABLE Department (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(30)
);

CREATE TABLE Employee (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept_id  INT
);

CREATE TABLE Staff (
    emp_id     INT PRIMARY KEY,
    emp_name   VARCHAR(30),
    manager_id INT
);
```

```
INSERT INTO Department VALUES (10, 'Sales'), (20, 'HR'), (30, 'IT'), (40, 'Finance');

INSERT INTO Employee VALUES
(1, 'Aisha', 10),
(2, 'Vivek', 20),
(3, 'Nisha', 30),
(4, 'Tarun', NULL);

INSERT INTO Staff VALUES
(1, 'Aisha', NULL),
(2, 'Vivek', 1),
(3, 'Nisha', 1),
(4, 'Tarun', 2);
```

Inner Join

QUERY

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
```

OUTPUT

emp_id	emp_name	dept_name
1	Aisha	Sales
2	Vivek	HR
3	Nisha	IT

Left Join

QUERY

```sql
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

OUTPUT

emp_id	emp_name	dept_name
1	Aisha	Sales
2	Vivek	HR
3	Nisha	IT
4	Tarun	NULL

Right Join

QUERY

```sql
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

OUTPUT

emp_id	emp_name	dept_name
1	Aisha	Sales
2	Vivek	HR
3	Nisha	IT
NULL	NULL	Finance

Fully Outer

QUERY

```sql
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

OUTPUT

emp_id	emp_name	dept_name
1	Aisha	Sales
2	Vivek	HR
3	Nisha	IT
4	Tarun	NULL
NULL	NULL	Finance

Cross Join

QUERY

```
SELECT e.emp_name, d.dept_name
FROM Employee e
CROSS JOIN Department d;
```

OUTPUT

emp_name	dept_name
Tarun	Sales
Nisha	Sales
Vivek	Sales
Aisha	Sales
Tarun	HR
Nisha	HR
Vivek	HR
Aisha	HR
Tarun	IT
Nisha	IT
Vivek	IT
Aisha	IT
Tarun	Finance
Nisha	Finance
Vivek	Finance
Aisha	Finance

Self-Join

QUERY

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Staff e
INNER JOIN Staff m ON e.manager_id = m.emp_id;
```

OUTPUT

employee	manager
Vivek	Aisha
Nisha	Aisha
Tarun	Vivek

LAB 8 | Group by, Having and Order by

Setup

```sql
CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

INSERT INTO Emp VALUES
(1, 'Aisha', 'Sales', 50000),
(2, 'Vivek', 'HR',    40000),
(3, 'Nisha', 'IT',    60000),
(4, 'Tarun', 'IT',    55000),
(5, 'Riya',  'Sales', 45000),
(6, 'Manoj', 'HR',    42000),
(7, 'Tara',  'IT',    65000);
```

Group by

QUERY

```sql
SELECT dept,
       COUNT(*)    AS total_emp,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary
FROM Emp
GROUP BY dept;
```

OUTPUT

dept	total_emp	total_salary	avg_salary
Sales	2	95000	47500.0000
HR	2	82000	41000.0000
IT	3	180000	60000.0000

Having

QUERY

```sql
SELECT dept, COUNT(*) AS total_emp
FROM Emp
GROUP BY dept
HAVING COUNT(*) > 2;
```

OUTPUT

dept	total_emp
IT	3

Order by

QUERY

```sql
SELECT emp_name, dept, salary
FROM Emp
ORDER BY salary DESC;
```

OUTPUT

emp_name	dept	salary
Tara	IT	65000
Nisha	IT	60000
Tarun	IT	55000
Aisha	Sales	50000
Riya	Sales	45000
Manoj	HR	42000
Vivek	HR	40000

Group by + Having + Order by

QUERY

```sql
SELECT dept, AVG(salary) AS avg_salary
FROM Emp
GROUP BY dept
HAVING AVG(salary) > 45000
ORDER BY avg_salary DESC;
```

OUTPUT

dept	avg_salary
IT	60000.0000
Sales	47500.0000

LAB 9 | Procedures

Basic Procedure - Setup

```
CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

INSERT INTO Emp VALUES
(1, 'Aisha', 'Sales', 50000),
(2, 'Vivek', 'HR',    40000),
(3, 'Nisha', 'IT',    60000),
(4, 'Tarun', 'IT',    55000),
(5, 'Riya',  'Sales', 45000),
(6, 'Manoj', 'HR',    42000),
(7, 'Tara',  'IT',    65000);
```

Basic Procedure

QUERY

```
DROP PROCEDURE IF EXISTS show_emp;

DELIMITER $$
CREATE PROCEDURE show_emp()
BEGIN
    SELECT * FROM Emp;
END$$
DELIMITER ;

CALL show_emp();
```

OUTPUT

emp_id	emp_name	dept	salary
1	Aisha	Sales	50000
2	Vivek	HR	40000
3	Nisha	IT	60000
4	Tarun	IT	55000
5	Riya	Sales	45000
6	Manoj	HR	42000
7	Tara	IT	65000

Procedure for Insert

QUERY

```
DELIMITER $$
CREATE PROCEDURE insert_emp(
    IN p_id     INT,
    IN p_name   VARCHAR(30),
    IN p_dept   VARCHAR(30),
    IN p_salary INT
)
BEGIN
    INSERT INTO Emp VALUES (p_id, p_name, p_dept, p_salary);
END$$
DELIMITER ;

CALL insert_emp(8, 'Akash', 'HR', 38000);
SELECT * FROM Emp;
```

OUTPUT

emp_id	emp_name	dept	salary
1	Aisha	Sales	50000
2	Vivek	HR	40000
3	Nisha	IT	60000
4	Tarun	IT	55000
5	Riya	Sales	45000
6	Manoj	HR	42000
7	Tara	IT	65000
8	Akash	HR	38000

Procedure for Update

QUERY

```
DELIMITER $$
CREATE PROCEDURE update_salary(
    IN p_id         INT,
    IN p_new_salary INT
)
BEGIN
    UPDATE Emp
    SET salary = p_new_salary
    WHERE emp_id = p_id;
END$$
DELIMITER ;

CALL update_salary(2, 45000);
SELECT * FROM Emp;
```

OUTPUT

emp_id	emp_name	dept	salary
1	Aisha	Sales	50000
2	Vivek	HR	45000
3	Nisha	IT	60000
4	Tarun	IT	55000
5	Riya	Sales	45000
6	Manoj	HR	42000
7	Tara	IT	65000

Procedure for Delete

QUERY

```
DELIMITER $$
CREATE PROCEDURE delete_emp(
    IN p_id INT
)
BEGIN
    DELETE FROM Emp
    WHERE emp_id = p_id;
END$$
DELIMITER ;

CALL delete_emp(5);
SELECT * FROM Emp;
```

OUTPUT

emp_id	emp_name	dept	salary
1	Aisha	Sales	50000
2	Vivek	HR	40000
3	Nisha	IT	60000
4	Tarun	IT	55000
6	Manoj	HR	42000
7	Tara	IT	65000

LAB 10 | Triggers

Setup

```
DROP TRIGGER IF EXISTS before_emp_insert;
DROP TRIGGER IF EXISTS after_emp_insert;
DROP TRIGGER IF EXISTS before_emp_update;
DROP TRIGGER IF EXISTS after_emp_update;
DROP TRIGGER IF EXISTS before_emp_delete;
DROP TRIGGER IF EXISTS after_emp_delete;

CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

CREATE TABLE Emp_log (
    log_id   INT AUTO_INCREMENT PRIMARY KEY,
    action   VARCHAR(30),
    emp_id   INT,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO Emp VALUES
(1, 'Aisha', 'Sales', 50000),
(2, 'Vivek', 'HR',    40000),
(3, 'Nisha', 'IT',    60000),
(4, 'Tarun', 'IT',    55000),
(5, 'Riya',  'Sales', 45000),
(6, 'Manoj', 'HR',    42000),
(7, 'Tara',  'IT',    65000);
```

Insert Triggers

```
DELIMITER $$

CREATE TRIGGER before_emp_insert
BEFORE INSERT ON Emp
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than 0';
    END IF;
END$$

CREATE TRIGGER after_emp_insert
AFTER INSERT ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('INSERT', NEW.emp_id);
END$$
```

Update Triggers

```
CREATE TRIGGER before_emp_update
BEFORE UPDATE ON Emp
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased';
    END IF;
END$$

CREATE TRIGGER after_emp_update
AFTER UPDATE ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('UPDATE', NEW.emp_id);
END$$
```

Delete Triggers

```
CREATE TRIGGER before_emp_delete
BEFORE DELETE ON Emp
FOR EACH ROW
BEGIN
    IF OLD.salary > 60000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete employee with salary above 60000';
    END IF;
END$$

CREATE TRIGGER after_emp_delete
AFTER DELETE ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('DELETE', OLD.emp_id);
END$$

DELIMITER ;
```

Testing the Triggers

QUERY

```
INSERT INTO Emp VALUES (8, 'Akash', 'HR', 38000);
INSERT INTO Emp VALUES (9, 'Disha', 'IT', 48000);
UPDATE Emp SET salary = 45000 WHERE emp_id = 2;
UPDATE Emp SET salary = 52000 WHERE emp_id = 1;
DELETE FROM Emp WHERE emp_id = 6;
DELETE FROM Emp WHERE emp_id = 5;

SELECT * FROM Emp_log;
```

OUTPUT

log_id	action	emp_id	log_time
1	INSERT	8	2026-10-08 20:28:46
2	INSERT	9	2026-10-08 20:28:46
3	UPDATE	2	2026-10-08 20:28:46
4	UPDATE	1	2026-10-08 20:28:46
5	DELETE	6	2026-10-08 20:28:46
6	DELETE	5	2026-10-08 20:28:46

Emp table after the statements above

QUERY

```
SELECT * FROM Emp;
```

OUTPUT

emp_id	emp_name	dept	salary
1	Aisha	Sales	52000
2	Vivek	HR	45000
3	Nisha	IT	60000
4	Tarun	IT	55000
7	Tara	IT	65000
8	Akash	HR	38000
9	Disha	IT	48000